



ADS IFBA

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

16 ANOS

TRABALHO PRÁTICO - IMPLEMENTAÇÃO COM MICROKERNEL

Tema: processamento de pedidos em e-commerce

Este trabalho prático deve ser desenvolvido em dupla, conforme orientação do professor. A solução deve estender o projeto microkernel fornecido, preservando sua estrutura principal e usando JavaFX para a interface gráfica. O objetivo é avaliar modelagem orientada a objetos, uso de interfaces, polimorfismo, composição, tratamento de exceções e separação de responsabilidades em uma aplicação de processamento de pedidos.

1. Contexto do Problema

Uma loja virtual precisa controlar produtos, estoque, carrinhos, pedidos, itens de pedido, formas de pagamento, políticas de desconto e métodos de entrega. O sistema deve permitir que diferentes estratégias de desconto, entrega e pagamento sejam escolhidas em tempo de execução, sem concentrar as regras de negócio em uma única classe.

As funcionalidades principais devem ser implementadas como plugins do microkernel. Extensões no kernel ou nas interfaces centrais devem ser feitas quando necessárias para expor serviços reutilizáveis aos plugins, mantendo compatibilidade com a arquitetura original.

2. Requisitos Funcionais

- Cadastrar, listar e consultar produtos com código, nome, descrição, preço unitário e quantidade disponível em estoque.
- Adicionar e remover produtos de um carrinho, validando a quantidade solicitada contra o estoque disponível.
- Criar um pedido a partir dos itens de um carrinho, representando cada produto selecionado como um item de pedido.
- Calcular o subtotal do pedido a partir da soma dos itens, considerando quantidade e preço unitário.
- Aplicar políticas de desconto selecionáveis, incluindo cupom promocional e desconto para estudante.
- Aplicar métodos de entrega selecionáveis, incluindo entrega padrão, entrega expressa e retirada sem custo.
- Processar pagamentos por cartão de crédito, pix e boleto, usando polimorfismo para encapsular as regras de cada forma de pagamento.
- Atualizar o status do pedido de acordo com o resultado do pagamento: pendente, pago, cancelado ou pagamento inválido.
- Reduzir o estoque apenas quando o pedido for confirmado conforme a regra definida pela equipe.
- Exibir no JavaFX uma tela ou aba que permita executar o fluxo mínimo: selecionar produtos, montar carrinho, escolher desconto, escolher frete, escolher pagamento e visualizar o total final.

3. Requisitos de Modelagem Orientada a Objetos

- Criar a interface/class Payable para representar formas de pagamento.
- Criar a interface/class DiscountPolicy para representar políticas de desconto.
- Criar a interface/class ShippingPolicy para representar políticas ou métodos de entrega.
- Implementar pelo menos três classes concretas de pagamento: CreditCardPayment, PixPayment e BoletoPayment.
- Implementar pelo menos duas classes concretas de desconto: CouponDiscountPolicy e StudentDiscountPolicy.
- Implementar pelo menos duas classes concretas de entrega: StandardShippingPolicy e ExpressShippingPolicy.
- Representar a composição entre Order e OrderItem: um pedido deve possuir uma coleção de itens de pedido.
- Separar classes de domínio, serviços de aplicação, classes de interface JavaFX, classes de persistência ou repositório e classes de integração com o microkernel.
- Evitar regras de negócio diretamente em manipuladores de eventos do JavaFX.
- Escrever o código-fonte, nomes de classes, métodos, atributos, pacotes, tabelas e colunas em inglês.

4. Exceções Obrigatórias

- Criar e usar uma exception para estoque insuficiente quando a quantidade solicitada for maior que a quantidade disponível.
- Criar e usar uma exception para pagamento inválido quando os dados ou o estado de pagamento impedirem a confirmação do pedido.
- Tratar as exceptions na camada adequada, apresentando mensagens compreensíveis na interface JavaFX sem encerrar a aplicação abruptamente.

5. Integração com Microkernel e Plugins

- A funcionalidade de e-commerce deve ser entregue como um ou mais plugins seguindo a convenção de pacotes do projeto fornecido.
- Cada plugin deve registrar seus menus, abas, ações e outros elementos de UI por meio do UIController exposto pelo core.
- Caso uma extensão no core seja necessária, ela deve ser justificada e documentada no vídeo de entrega.
- A aplicação deve continuar executando com o comando Maven indicado no projeto base.
- O uso de JavaFX é obrigatório para a interface gráfica.

6. Banco de Dados de Apoio

Foi disponibilizado um ambiente MariaDB com dados iniciais do domínio de e-commerce. A equipe pode usar esses dados como apoio para persistência, testes manuais ou demonstração. O uso do banco deve respeitar nomes em inglês para tabelas, colunas e valores técnicos.

- O banco contém exemplos de customers, products, stock_movements, carts, cart_items, orders, order_items, payments, discounts, order_discounts e shipping_methods.
- Os dados incluem pagamentos por credit card, pix e boleto, descontos por coupon e student discount, além de cenários de pagamento aprovado, pendente e falho.

- O uso do banco não substitui a necessidade de modelagem orientada a objetos correta dentro da aplicação Java.

7. Cálculo Obrigatório do Total do Pedido

A solução deve demonstrar, no código e na interface, o cálculo do total final de um pedido usando estratégias diferentes. O cálculo mínimo esperado é:

total final = subtotal dos itens - descontos aplicados + custo de entrega

O subtotal deve ser calculado pelos itens do pedido. O desconto deve ser calculado por objetos que implementem DiscountPolicy. O frete deve ser calculado por objetos que implementem ShippingPolicy. O pagamento deve ser processado por objetos que implementem Payable.

8. Entregáveis

- Código-fonte completo do projeto, com os plugins implementados, **sem arquivos .class**.
- Link do vídeo gravado **pela dupla**, com no máximo 10 minutos, explicando:
 1. Descrição das principais decisões, classes e responsabilidades.
 2. Indicação das interfaces, classes polimórficas, exceptions e pontos de composição usados na solução.

9. Critérios de Avaliação - 10,0 pontos

- 1,5 ponto: modelagem correta de produtos, carrinho, pedido, itens de pedido, pagamento, desconto, entrega e estoque.
- 1,5 ponto: uso correto de interfaces, classes concretas e polimorfismo em Payable, DiscountPolicy e ShippingPolicy.
- 1,0 ponto: uso correto de composição entre Order e OrderItem, sem duplicação desnecessária de responsabilidades.
- 1,0 ponto: tratamento correto de estoque insuficiente e pagamento inválido com exceções apropriadas.
- 1,0 ponto: cálculo correto do subtotal, descontos, frete e total final do pedido.
- 1,0 ponto: integração adequada com o microkernel por meio de plugins e, quando necessário, extensões justificadas no core.
- 1,0 ponto: interface JavaFX funcional, organizada e coerente com o fluxo de processamento de pedidos.
- 1,0 ponto: organização, legibilidade, nomes em inglês no código-fonte e separação entre domínio, UI, serviços e persistência.

10. Restrições

- Não alterar a proposta arquitetural do microkernel sem justificativa técnica.
- Não usar apenas condicionais para substituir o polimorfismo exigido.
- Não escrever código-fonte, nomes de banco, tabelas ou colunas em português.
- Não entregar apenas telas sem regras de domínio implementadas.

11. Forma de Entrega

Enviar arquivo zipado para sandroandrade@ifba.edu.br com subject "INF008 – <nomes dos alunos>"

Prazo final (inadiável): 23/07/2026